

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 8

Operating Systems

Topic: The Same Calls, Inside Your Own OS

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Purpose of the Work 3

2 Part 1 – `run`, Inside xv6 3

2.1 Observations 4

3 Part 2 – `pingpong` 6

3.1 Observations 7

4 Conclusion 8

1. Purpose of the Work

The purpose of this laboratory work is to repeat inside xv6 the same basic process operations used on Linux in the previous laboratory. I used `fork`, `exec` and `wait` to create a child process and replace it with another program, then used two pipes to make a parent and child exchange one byte in both directions. This made it possible to directly compare the Linux interface with the much simpler xv6 implementation.

2. Part 1 – `run`, Inside xv6

The first program uses the same three steps as the Linux program from Lab 7: fork a child, execute another program inside the child, then wait for that child in the parent.

user/run.c

```
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6: echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                    // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                        // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                // step 2: become echo
    printf("exec failed\n");           // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                       // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

With `echo`, the child successfully became the program and printed the supplied arguments.

Output – `echo`

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

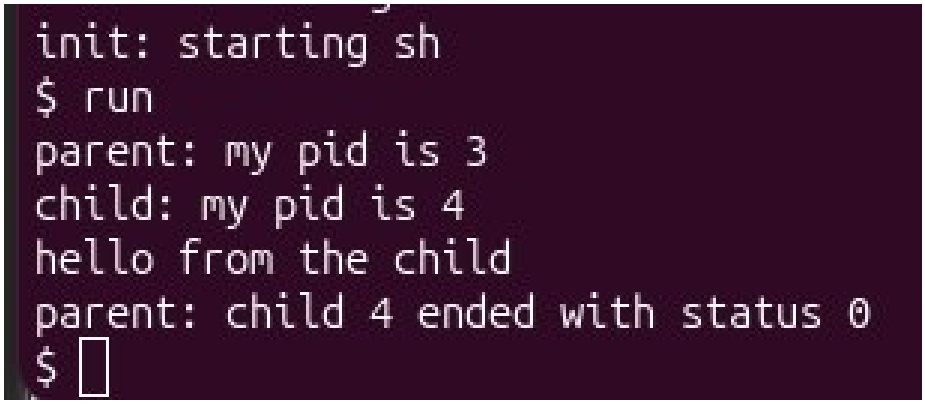


Figure 1.1 – xv6 `run` executing `echo`.

I then changed the argument list and `exec` call so that the child became `ls` instead.

Relevant change

```
char *args[] = { "ls", 0 };
exec("ls", args);
```

The listing was printed by `ls`, and after it finished the parent received status 0.

Finally, I tried to execute a program which does not exist. Since `exec` failed, the code after it was reached and the child exited with status 1.

Output – failed `exec`

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
$
```

2.1. Observations

1. What is the same between the Linux and xv6 versions?

The program in Linux and xv6 are almost identical. The same three basic calls are used: `fork`, `exec` and `wait`. The only thing that is changed is the implementation of `exec` and `wait`, which in the xv6 OS is simplified. On Linux I used `execlp` and `waitpid`, while xv6

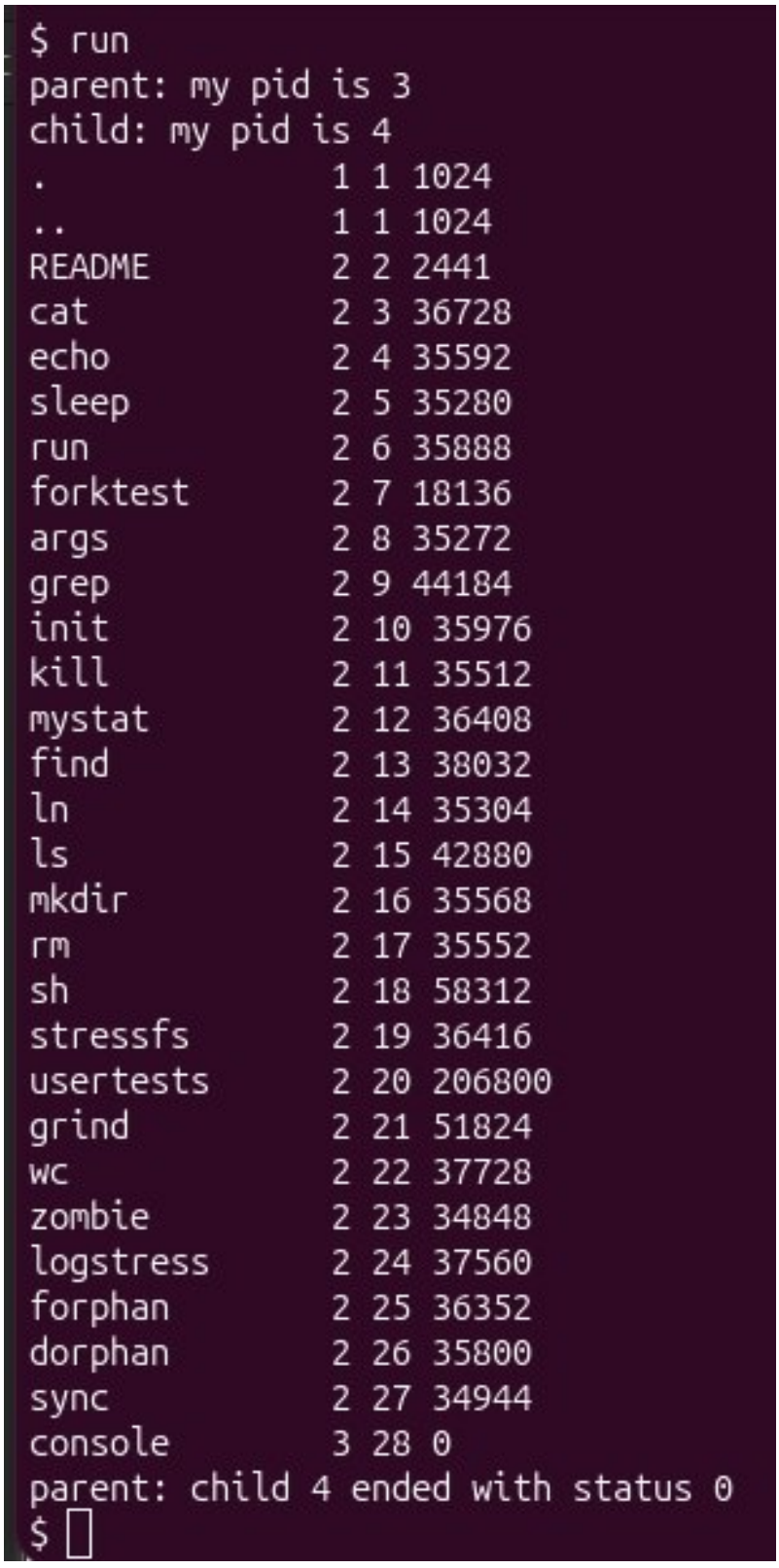


Figure 1.2 – The child process replaced by `ls`.

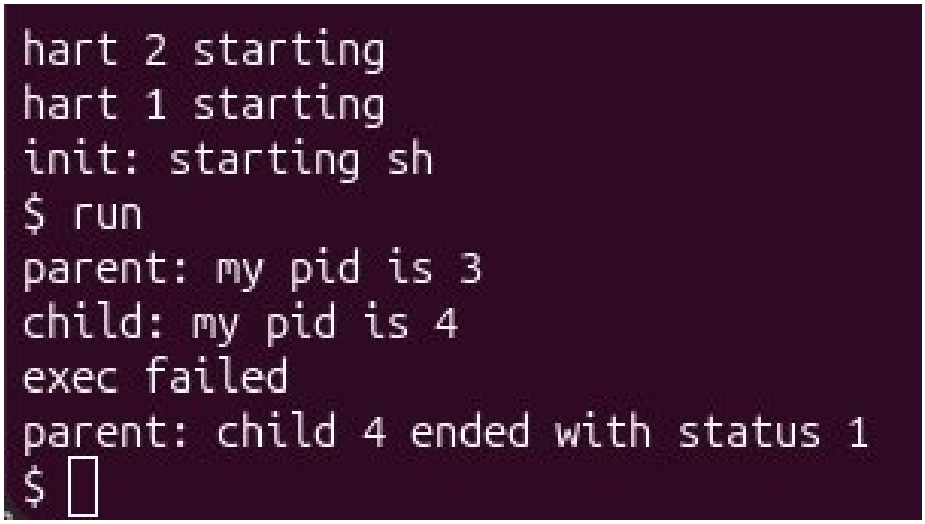


Figure 1.3 – Failed `exec` in xv6 and status 1.

exposes the simpler `exec` and `wait` calls.

2. Who prints "hello from the child": `run` or `echo`?

`run.c` forks, creating a new process which executes `echo`, passed with the arguments of what has to be written on screen. Hence `echo` is the program which actually writes `hello from the child` to the screen. Once `exec` succeeds, the child is no longer running the original `run` code.

3. Why is the failure status 1 in xv6 but 127 on Linux?

On Linux return value 127 means command not found. However xv6 is primitive enough to the point it does not have the same specific return value convention for all situations, hence a 1 to mean something went wrong is good enough. In this program the difference is also directly caused by the code after the failed `exec`: the Linux version used `exit(127)`, while the xv6 version uses `exit(1)`.

3. Part 2 – `pingpong`

For the second part I made two processes communicate in both directions. The parent first sends one byte to the child. The child reads it, prints the ping message, writes the same byte back through another pipe and exits. The parent reads the returned byte, prints pong, and waits for the child.

user/pingpong.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
```

```
  char b = 'e';

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    // DONE: read 1 byte from p2c[0] into b
    read(p2c[0], &b, 1);
    printf("%d: received ping, data: %c\n", getpid(), b);
    // DONE: write the byte b into c2p[1]
    write(c2p[1], &b, 1);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  // DONE: write 1 byte into p2c[1]
  write(p2c[1], &b, 1);
  // DONE: read 1 byte from c2p[0] into b
  read(c2p[0], &b, 1);
  printf("%d: received pong, data: %c\n", getpid(), b);
  wait(0);
  exit(0);
}
```

The program worked as expected. In my run the child had PID 4 and the parent PID 3, and both sides received the same byte, `e`.

Output

```
$ pingpong
4: received ping, data: e
3: received pong, data: e
$
```

3.1. Observations

1. Why are two pipes needed instead of one?

A pipe has an input and an output, but it is a one-direction communication channel. You cannot use one pipe cleanly for both directions because each process would need to act as both sender and receiver on the same channel. Using two pipes makes the direction obvious: one is parent-to-child and the other is child-to-parent.

2. Which call makes the parent wait for the child to reply?

The `read` in the parent is the call that waits. It blocks until data becomes available on the read end of the child-to-parent pipe. In my code, after the child writes the byte back it exits, and the parent can read the returned byte and continue. The later `wait(0)` is used to collect the child after the communication is finished.



Figure 2.1 – Parent and child exchanging one byte through two pipes.

4. Conclusion

This laboratory showed that the main process operations used on Linux also exist inside xv6, even though xv6 exposes them in a much simpler form. The `run` program made the `fork`–`exec`–`wait` sequence very visible: the parent creates a child, the child becomes another program, and the parent receives its exit status. The `pingpong` program then showed how processes can synchronize naturally through blocking reads and communicate using pipes. Overall, this lab connected the Linux experiments from Lab 7 with the same mechanisms inside the operating system we are building.